

Migrations SQL

avec Supabase — Guide pratique Social Seller

Djiguitech · Social Seller SaaS | Stack : PostgreSQL 15 · Supabase · RLS

1. Pourquoi des migrations ?

Une migration est un fichier SQL versionné décrivant une modification de schéma. Plutôt que de modifier la base manuellement dans l'UI Supabase, on préfère ces fichiers car :

- **Reproductibilité** — dev, staging et prod ont exactement le même schéma.
- **Traçabilité** — le fichier est dans Git, chaque changement est daté et signé.
- **Rollback possible** — on peut écrire une migration inverse si besoin.

■ Convention Social Seller : numérotation 001_, 002_... dans `supabase/migrations/`.

2. Structure d'une migration

Chaque fichier suit le même squelette :

```
-- Commentaire décrivant l'objectif

-- 1. Création de table
create table public.ma_table (
  id          uuid primary key default gen_random_uuid(),
  tenant_id   uuid not null references public.tenants(id),
  nom         text not null,
  created_at  timestampz not null default now(),
  updated_at  timestampz not null default now(),
  deleted_at  timestampz          -- soft delete
);

-- 2. Index partiel (actifs seulement)
create index ma_table_tenant_idx on public.ma_table (tenant_id)
  where deleted_at is null;

-- 3. Trigger updated_at
```

```
create trigger ma_table_set_updated_at
  before update on public.ma_table
  for each row execute function public.set_updated_at();

-- 4. RLS
alter table public.ma_table enable row level security;
revoke all on public.ma_table from anon, authenticated;
```

■ Ne jamais désactiver RLS en production — règle absolue Social Seller.

3. Opérations DDL courantes

3.1 Ajouter une colonne

```
alter table public.orders
  add column if not exists tracking_code text;
```

■ Utiliser IF NOT EXISTS pour éviter l'erreur si la migration est rejouée.

3.2 Renommer une colonne

```
alter table public.products rename column qty to stock_quantity;
```

■ Breaking change : mettre à jour le code applicatif avant de déployer.

3.3 Changer le type d'une colonne

```
alter table public.orders
  alter column total_amount type numeric(12,2)
  using total_amount::numeric(12,2);
```

3.4 Contrainte CHECK

```
alter table public.customers
  add constraint customers_source_check
  check (source in ('whatsapp', 'facebook', 'tiktok', 'manual'));
```

3.5 Clé étrangère optionnelle

```
alter table public.conversations
  add column if not exists customer_fk_id uuid
  references public.customers(id);
```

■ FK nullable = relation optionnelle. Une conversation peut exister sans client enregistré.

4. Index — quand et comment

Un index accélère les SELECT mais ralentit INSERT/UPDATE. Indexer les colonnes utilisées dans WHERE et JOIN fréquents.

Index partiel (actifs seulement)

```
create index conversations_tenant_idx
  on public.conversations (tenant_id)
  where deleted_at is null; -- taille réduite, maintien moins coûteux
```

Index unique (contrainte métier)

```
create unique index customers_tenant_external_id_idx
on public.customers (tenant_id, external_id)
where external_id is not null and deleted_at is null;
```

■ Index unique partiel : plusieurs NULL autorisés car NULL != NULL en SQL.

Index composite (deux colonnes souvent filtrées ensemble)

```
create index messages_conv_created_idx
on public.messages (conversation_id, created_at desc);
```

5. Triggers

Un trigger exécute automatiquement une fonction PL/pgSQL avant/après une opération DML. Social Seller utilise deux patterns.

5.1 updated_at automatique

La fonction est créée une seule fois et réutilisée par tous les triggers :

```
create or replace function public.set_updated_at()
returns trigger language plpgsql as $$
begin
    new.updated_at = now();
    return new;
end;
$$;

create trigger ma_table_set_updated_at
before update on public.ma_table
for each row execute function public.set_updated_at();
```

5.2 Synchronisation auth.users → users (onboarding)

```
create or replace function public.handle_new_user()
returns trigger language plpgsql security definer as $$
begin
    insert into public.users (id, email, full_name)
    values (new.id, new.email,
           new.raw_user_meta_data->>'full_name');
    return new;
end;
$$;

create trigger on_auth_user_created
after insert on auth.users
for each row execute function public.handle_new_user();
```

■ SECURITY DEFINER : la fonction s'exécute avec les droits de son propriétaire. Ne jamais exposer raw_user_meta_data directement.

6. Row Level Security (RLS)

RLS est la première ligne de défense multi-tenant. Le backend accède via service_role (bypass RLS natif). Aucun rôle anon/authenticated ne touche directement les tables métier.

Pattern Social Seller — service_role only

```
alter table public.customers enable row level security;
revoke all on public.customers from anon, authenticated;
-- service_role bypasses RLS natively, no policy needed.
```

Pattern alternatif — accès direct depuis le mobile

```
create policy "tenant members read own messages"
on public.messages for select
using (
  tenant_id = (
    select tenant_id from public.users
    where id = auth.uid()
  )
);
```

■ Accès client mobile via SDK Supabase → écrire une policy SELECT. Backend seul → revoke all.

7. Soft Delete

On ne supprime jamais une ligne en production : on pose un timestamp `deleted_at`. Cela préserve les FK, les logs d'audit et permet un recovery.

Schéma

```
deleted_at timestampz -- null = actif, non-null = supprimé
```

Requêtes côté backend (Supabase JS)

```
// Lister les actifs
supabase.from('customers').select('*').is('deleted_at', null)

// Suppression logique
supabase.from('customers')
  .update({ deleted_at: new Date().toISOString() })
  .eq('id', id).eq('tenant_id', tenantId)
```

8. Workflow d'application d'une migration

1. Écrire le fichier SQL dans `supabase/migrations/NNN_description.sql`
2. Ouvrir Supabase Dashboard → SQL Editor → New query
3. Coller le contenu et cliquer Run
4. Vérifier le retour : *Success. No rows returned* = OK
5. Commit Git : `git add . && git commit -m 'migration: description'`

■ Toujours tester sur dev/staging avant d'appliquer en production.

Erreurs fréquentes

Erreur	Cause	Solution
function moddatetime() does not exist	Extension non installée	Écrire set_updated_at() manuellement
column already exists	Migration rejouée	ADD COLUMN IF NOT EXISTS
relation does not exist	Table absente / mauvais ordre	Vérifier l'ordre des migrations
violates foreign key constraint	Données orphelines	Nettoyer les données d'abord
permission denied for table	RLS bloquant	Utiliser la service_role key

9. Checklist avant toute migration

- La table a un tenant_id NOT NULL avec FK vers tenants(id)
- RLS activé + revoke all (ou policies si accès client)
- Soft delete : colonne deleted_at présente
- Index partiel WHERE deleted_at is null sur tenant_id
- Trigger updated_at si la table est modifiable
- Unicité métier couverte par un index unique
- Pas de DROP TABLE / COLUMN sans migration de rollback
- ADD COLUMN sur table existante en prod : utiliser IF NOT EXISTS
- Renommage : déployer le code applicatif AVANT la migration
- Jamais ALTER TABLE ... DISABLE ROW LEVEL SECURITY en production